Влияние нормализации и метода главных компонент на точность машинного обучения

Теоретические основы

При машинном обучении модель больше «обращает внимания» на признаки с большими значениями, тем самым малые значения учитываются меньше. Если в обучающих данных велика разница по диапазону значений между признаками (столбцами), точность может быть неудовлетворительной. Для решения проблемы данные сжимаются/растягиваются до одного диапазона так, чтобы не было указанного дисбаланса признаками. Описанная процедура называется нормализацией. Нормализация – приведение всех значений признаков к одному диапазону (как правило, 0…1). Существуют различные виды нормализации, в данной работе используется Z-Score (вызывается кодом StandardScaler).

Некоторые признаки могут иметь малую информативность из-за плохой проекции (данные разных классов по этому признаку накладываются, перемешиваются). Для улучшения информативности можно воспользоваться другими проекциями, для чего попытаться подобрать ее поворотом пространства признаков («взглянуть» с другого «ракурса» на данные так, чтобы они меньше смешивались).

При повороте, говоря грубо, координаты поворачиваемых точек умножаются на косинусы и синусы угла поворота, то есть, поворот проекции признаков означает вычисление нового признака из исходных. Один из распространенных методов изменения проекции признаков – метод главных компонент (МГК или PCA). По МГК подбирается проекция, в которой максимизируется дисперсия (максимизируется расстояние между классами). Полученные проекции признаков после МГК именуют компонентами, они могут иметь большую информативность (зачастую, лишь некоторые из них).

Степень информативности компонентов принято оценивать по доле объясненной или совокупной дисперсии между классами – отбирается столько первых компонентов, для которых доля

совокупной дисперсии близка к порогу (как правило, берут 0,95 и выше). Можно судить и по объясненной дисперсии – компонент, после которого доля объясненной дисперсии крайне мала считается последним информативным. Тем самым, возможно уменьшить число признаков, что может быть использовано как для сжатия, так и для выявления наиболее информативных признаков (в некоторых случаях МГК повышает точность).

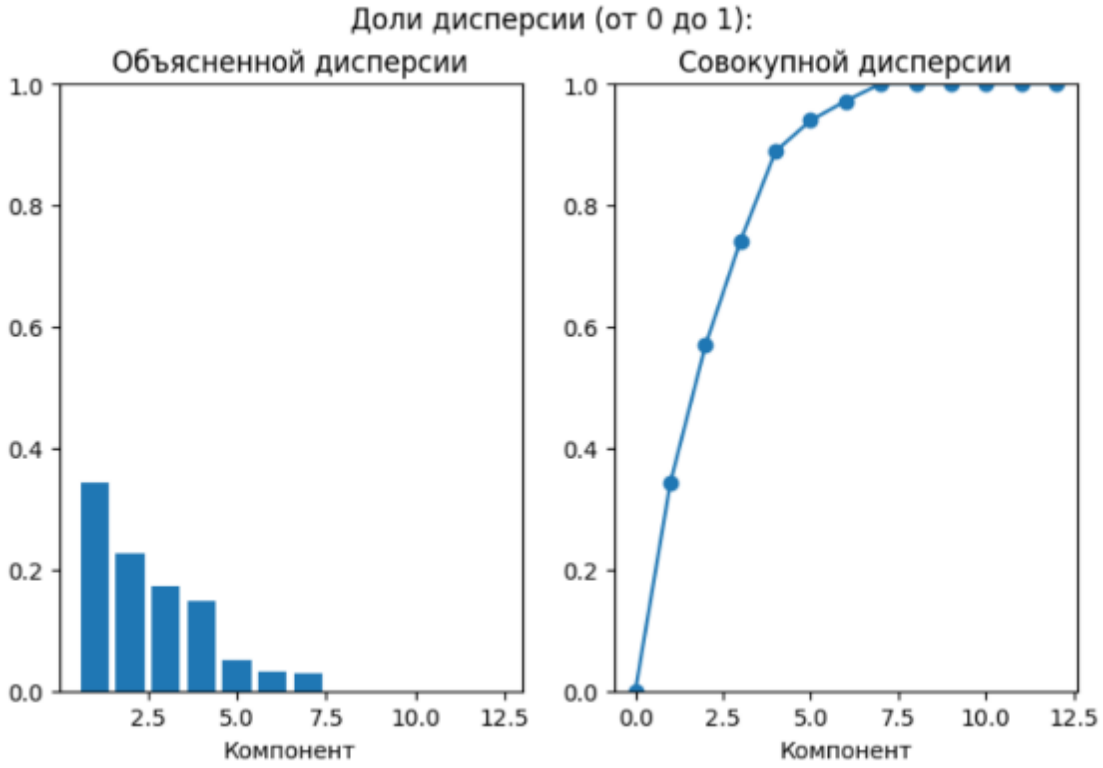

Рис. 10.2. Доли дисперсий

С помощью МГК можно менять размерность полученных данных от 1 до числа признаков в исходных данных. Другой вопрос в эффективности метода – дело в том, что МГК повышает точность машинного обучения только для данных, в которых присутствует зависимость между признаками, то есть информация частично между ними дублируется. МГК, как правило, работает значительно лучше при предварительной нормализации данных.

Самостоятельное задание

В данной работе необходимо рассмотреть эффективность нормализации и МГК для регрессии и классификации на примере трех датасетов. Для этого приведен пример кода, из которого следует собирать по частям собственный код, в зависимости от операции над датасетом (регрессия или классификация).

1) Импортируйте датасет:

1.1) в пустой ячейке введите и выполните (после выполнения, – вывода текста, – удалите ячейку):

pip install seaborn scikit-learn numpy matplotlib pandas

1.2) скопируйте себе все ячейки примера кода (табл. 10.1).

Таблица 10.1

Исходный код

Исходный код для работы – Python	

1	`from sklearn.svm import SVR`
	`from sklearn.linear_model import LogisticRegression`
	`import pandas as pd`
	`from sklearn.model_selection import train_test_split`
	`import matplotlib.pyplot as plt`
	`import numpy as np`
	`import warnings`
	`import seaborn as sns`
	`from sklearn.exceptions import ConvergenceWarning, DataConversionWarning`
	`from sklearn import preprocessing`
	`from sklearn.decomposition import PCA`
	`from sklearn.metrics import accuracy_score`
	`warnings.filterwarnings('ignore', category=ConvergenceWarning)`
	`warnings.filterwarnings('ignore', category=DataConversionWarning)`
	`def delta_plot(X_test, y_test, reg):`
	`  y_pred = reg.predict(X_test)`
	`  x = range(len(y_pred))`
	`  delta = list(abs(y_test-y_pred)/max(y_test)*100)`
	`  mx, mn = max(delta), sum(delta)/len(delta)`
	`  plt.plot(x,y_pred)`
	`  plt.plot(x,y_test)`
	`  print(f'Максимальная погрешность:{mx}%, средняя:{mn}%')`
	`  plt.ticklabel_format(useOffset=False, style='plain')`
	`  plt.xlabel('номер элемента данных')`
	`  plt.ylabel('Цена, млн.у.е')`
	`  plt.legend(['предсказанная', 'истинная'])`
	`  plt.show()`

Продолжение таблицы 10.1

Исходный код для работы – Python	

2	`def fit_classes(X_tr, y_tr, X_te, y_te, clf):`
	`    X_tr, y_tr = np.array(X_tr), np.array(y_tr)`
	`    X_te, y_te = np.array(X_te), np.array(y_te)`
	`    clf.fit(X_tr,y_tr)`
	`    acc = round(accuracy_score(y_te, clf.predict(X_te)),7)`
	`    print(f'Точность класификации: {acc}')`
	`def plot_variance(pca, figwidth=8, dpi=100):`
	`    fig, axs = plt.subplots(1, 2)`
	`    n = pca.n_components_`
	`    grid = np.arange(1, n + 1)`
	`    evr = pca.explained_variance_ratio_`
	`    axs[0].bar(grid, evr)`
	`    label, title = 'Компонент', 'Объясненной дисперсии'`
	`    axs[0].set(xlabel=label, title=title, ylim=(0.0, 1.0))`
	`    cv = np.cumsum(evr)`
	`    axs[1].plot(np.r_[0, grid], np.r_[0, cv], "o-")`
	`    title = 'Совокупной дисперсии'`
	`    axs[1].set(xlabel=label, title=title, ylim=(0.0, 1.0))`
	`    fig.suptitle('Доли дисперсии (от 0 до 1):')`
	`    fig.set(figwidth=figwidth, dpi=dpi)`
	`    plt.show()`

3	`???`
	`df = pd.read_csv(link).dropna()`
	`X = df.drop(y_name, axis='columns')`
	`Y = df[y_name]`
	`X_train, X_test, y_train, y_test = train_test_split(X, Y, train_size=0.67,`
	`random_state=42)`
	`df`

1.2) При копировании кода могут теряться отступы, воспроизведите отступы в соответствии с теми, что присутствуют в табл. 10.1;

1.3) выполните первые 2 ячейки из примера кода, они подготовят необходимые для работы функции;

1.4) в ячейке 3, в начале (в новой пустой строке), вставьте вместо ??? переменные link и y_name (соответственно ссылка на датасет и имя выходной переменной в нем) для первого датасета (сет 1 в табл. 10.2), значения переменных link и y_name для различных датасетов ниже.

Housing (регрессия). 12 признаков

данные о наполнении, площади домов и цены,

за которые эти дома были проданы

link = 'https://raw.githubusercontent.com/heymynameis/Datasets-for-lab/main/Housing.csv'

y_name = 'price'

Telecom (классификация). 19 признаков

данные о пользователях сотового оператора

с указанием, ушел ли клиент от оператора

link = 'https://raw.githubusercontent.com/heymynameis/Datasets-for-lab/main/telecom.csv'
y_name = 'Churn'

Bank (классификация). 16 признаков

данные различных статистик для банков

с указанием, подверглись ли они банкротству

link = 'https://raw.githubusercontent.com/heymynameis/Datasets-for-lab/main/bank.csv'
y_name = 'Bankrupt?'

Carprice (регрессия). 25 признаков

данные о наполнении, площади домов и цены,

за которые эти дома были проданы

link = 'https://raw.githubusercontent.com/heymynameis/Datasets-for-lab/main/carprice.csv'
y_name = 'price'

Mobile User (классификация). 6 признаков

данные о классе поведения людей (агрессия и т.п.)

в зависимости от параметров использования ими смартфонов

link ='https://raw.githubusercontent.com/heymynameis/Datasets-for-lab/refs/heads/main/mobile.csv'
y_name = 'User Behavior Class'

Таблица 10.2

Варианты заданий

Вар.	сет 1	сет 2	Вар.	сет 1	сет 2
1	Housing	Bank	16	Mobile User	Carprice
2	Carprice	Telecom	17	Housing	Bank
3	Bank	Housing	18	Carprice	Telecom
4	Mobile User	Carprice	19	Bank	Housing
5	Housing	Mobile User	20	Telecom	Carprice
6	Carprice	Telecom	21	Housing	Mobile User
7	Bank	Housing	22	Carprice	Mobile User
8	Telecom	Carprice	23	Bank	Housing
9	Housing	Mobile User	24	Telecom	Carprice
10	Mobile User	Carprice	25	Housing	Bank
11	Bank	Housing	26	Carprice	Telecom
12	Telecom	Carprice	27	Mobile User	Housing
13	Housing	Bank	28	Mobile User	Carprice
14	Carprice	Telecom	29	Housing	Bank
15	Mobile User	Housing	30	Carprice	Telecom

1.5) Выполните полученную ячейку, выведется таблица данных датасета. Если в ней в качестве значений присутствуют не числовые значения (дата, текст и т.д.), необходимо их заменить числами, так как любая модель машинного обучения может работать лишь с числами (целыми или дробными). Для этой замены скопируйте в ту же ячейку перед строкой X = df.drop(y_name, axis='columns') (в новой пустой строке) код для замены (см. табл. 10.3, ячейка 1) и выполните ячейку, выведется обновленный датасет. Если в датасете только числа – пропустите данный пункт.

Таблица 10.3

Код для замены

	Исходный код для работы – IPython

1	`types = ['int','int64','int32','float','float32','float64']`
	`tch = lambda c: any([str(c.dtype)==t for t in types])`
	`for col in df.columns:`
	`  if not tch(df[col]):`
	`  df[col] = df[col].astype(str)`
	`  for i, uniq in enumerate(df[col].unique()):`
	`  df[col] = df[col].replace(uniq,str(i))`
	`  df[col] = df[col].astype(int)`

2	`def min_pow(x):`
	`  for n in range(10):`
	`  if x-x%(10**n) == 0: return x*10**-(n)`
	`df[y_name] = df[y_name].apply(min_pow)`

1.6) Обратите внимание на столбец с именем, совпадающим с y_name для вашего варианта в выводимой таблице: если видимые числа в нем превышают 100 – скопируйте себе код из ячейки 2 табл. 10.3 перед строкой X = df.drop(y_name, axis='columns') (в новой пустой строке). Используемый в работе классификатор плохо обрабатывает данные, в которых результат предсказаний (столбец y_name) существенно превосходит значения в столбцах признаков (у всех вариантов признаки в диапазоне до 100), а код из табл. 10.3, ячейки 2 предварительно уменьшает данные в 10 в некоторой степени раз.

2) Осуществите регрессию/классификацию:

2.1) если выше в скобках от имени вашего датасета указана регрессия, то скопируйте себе 1 ячейку из табл. 10.4, если классификация – 2.

Примечание: в этой работе для классификации используется классификатор на основе логистической регрессии, а для регрессии – модель на основе метода опорных векторов. Обе модели просты, но достаточно эффективны для табличных данных.

Таблица 10.4

Ячейки для регрессии/классификации

Исходный код для работы – Python	
1	X_train_in = ??? X_test_in = ??? reg = SVR() reg.fit(X_train_in, y_train) delta_plot(X_test_in, y_test, reg)
2	X_train_in = ??? X_test_in = ??? clf = LogisticRegression() fit_classes(X_train_in, y_train, X_test_in, y_test, clf)

2.2) В скопированной ячейке замените ??? на имена массивов входных данных – для исходных данных (на данном этапе подставляются именно они) это будут X_train и X_test;

2.3) если выше в скобках от имени вашего датасета указана регрессия, то пропустите данный пункт, если классификация – скопируйте себе из табл. 10.5 код вниз текущей ячейки. Этот код нужен для попарного отображения всех признаков, можно видеть, накладываются ли данные разных классов по каждой из пар признаков. В работе ограничено число признаков графика до 5 (если их меньше – выведется меньше, если больше – только 5), чтобы время выполнения кода было приемлемым.

Таблица 10.5

Код для вывода графика признаков

Исходный код для работы – Python

```
x_data = X_test_in
if type(x_data) == pd.core.frame.DataFrame:
    x_data = x_data.values
if x_data.shape[1]>5:
    x_data = x_data[:, : 5]
fet = range(x_data.shape[1])
dataset = pd.DataFrame(data=x_data, columns=[df.columns[i] for i in fet])
dataset['out'] = y_test.values
sns.pairplot(dataset, hue='out')
plt.show()
```

2.4) Выполните ячейку, выведется график погрешности для регрессии или значение точности для данных без предобработки с графиком распределения признаков.

3) Проверьте влияние нормализации на точность регрессии/классификации:

3.1) скопируйте себе ячейку кода из табл. 10.6, реализующую нормализацию.

Таблица 10.6

Код для нормализации

Исходный код для работы – Python

`sc = preprocessing.StandardScaler()`
`Xs_train = sc.fit_transform(X_train)`
`Xs_test = sc.fit_transform(X_test)`

3.2) Если выше в скобках от имени вашего датасета указана регрессия, то скопируйте вниз ячейки с нормализацией код 1 ячейки из табл. 10.4, если классификация – код ячейки 2 из той же таблицы (подставив Xs_train и Xs_test вместо ???). Также для датасетов с классификацией нужно скопировать код из табл. 10.5, вставив его вниз той же ячейки. Выполните ячейку, выведется график погрешности для регрессии или значение точности для данных после нормализации вместе с графиком признаков.

4) Проверьте влияние МГК на точность регрессии/классификации:

4.1) скопируйте в новую ячейку код из табл. 10.7, который реализует МГК (без нормализации), замените в ней M (размерность) на число признаков датасета (указан выше рядом с названием датасета).

Таблица 10.7

Код для МГК

Исходный код для работы – Python

`pca = PCA(n_components = M)`
`Xpca_train = pca.fit_transform(X_train)`
`Xpca_test = pca.transform(X_test)`

4.2) Если выше в скобках от имени вашего датасета указана регрессия, то скопируйте вниз ячейки с МГК код 1 ячейки из табл. 10.4, если классификация – код ячейки 2 из той же таблицы (подставив Xpca_train и Xpca_test вместо ???). Также для датасетов с классификацией нужно скопировать код из табл. 10.5, вставив его вниз той же ячейки. Выполните ячейку, выведется график погрешности для регрессии или значение точности для данных после применения МГК вместе с графиком распределения признаков;

4.3) после ячейки, полученной в п. 4.2 вставьте новую (пустую), в которой впишите plot_variance(pca) и выполните ячейку, выведутся графики совокупной и объясненной дисперсии. Само число компонент для МГК можно менять в диапазоне от 1 до числа признаков датасета, но по определению оптимальным можно считать такое число компонент, при котором доля совокупной дисперсии достигает 1.0 (на вашем графике справа). Задайте это число вместо заданного изначально в ячейке по п. 4.2 (если 1.0 на правом графике достигается только при последнем компоненте – оставляйте как есть).

5) Проверьте влияние комбинации нормализации и МГК на точность регрессии/классификации:

5.1) скопируйте в новую ячейку код из табл. 10.8, который реализует МГК для данных после нормализации, замените в ней M (размерность) на число признаков датасета (указан выше рядом с названием датасета).

Таблица 10.8

Код для МГК после нормализации

Исходный код для работы – Python

`pca = PCA(n_components = M)`
`Xspca_train = pca.fit_transform(Xs_train)`
`Xspca_test = pca.transform(Xs_test)`

5.2) Если выше в скобках от имени вашего датасета указана регрессия, то скопируйте вниз ячейки с МГК после нормализации код 1 ячейки из табл. 10.4, если классификация – код ячейки 2 из той же таблицы (подставив Xspca_train и Xspca_test вместо ???). Также для датасетов с классификацией нужно скопировать код из табл. 10.5,

вставив его вниз той же ячейки. Выполните ячейку, выведется график погрешности для регрессии или значение точности для данных после применения МГК и нормализации вместе с графиком распределения признаков;

5.3) после ячейки, полученной в п. 5.2 вставьте новую (пустую), в которой впишите plot_variance(pca) и выполните ячейку, выведутся графики совокупной и объясненной дисперсии. Задайте в ячейке по п. 5.2 оптимальное число компонент, при котором доля совокупной дисперсии достигает 1.0 (на вашем графике справа) вместо заданного изначально (если 1.0 на правом графике достигается только при последнем компоненте – оставляйте как есть).

6) Обработайте второй датасет:

6.1) скопируйте себе еще раз ячейку 3 из табл. 10.1, в ее начале (в новой пустой строке) задайте переменные link и y_name (соответственно ссылка на датасет и имя выходной переменной в нем) для второго датасета (сет 2 в табл. 10.2), значения переменных link и y_name для различных датасетов выше, выполните ячейку – выведется таблица данных второго датасета;

6.2) выполните ячейку, выведется таблица с данными. Если в ней в качестве значений присутствуют не числовые значения (дата, текст и т.д.), необходимо их заменить числами, так как любая модель машинного обучения может работать лишь с числами (целыми или дробными). Для этой замены скопируйте в ту же ячейку перед строкой X = df.drop(y_name, axis='columns') (в новой пустой строке) код для замены (см. табл. 10.3, ячейка 1) и выполните ячейку, выведется обновленный датасет. Если в датасете только числа – пропустите данный пункт;

6.3) обратите внимание на столбец с именем, совпадающим с y_name для вашего варианта в выводимой таблице: если видимые числа в нем превышают 100 – скопируйте себе код из ячейки 2 табл. 10.3 перед строкой X = df.drop(y_name, axis='columns') (в новой пустой строке);

6.4) повторите п. 2.1-5.3 для этого датасета.